

Unconditional Branch (GOTOnnn and M99 Pnnnn)

In the Haas control, there are two methods of branching unconditionally. An unconditional branch will always branch to a specified block. `M99 P15` will branch unconditionally to block number 15. The `M99` can be used whether or not macros is installed and is the traditional method for branching unconditionally in the Haas control. `GOTO15` does the same as `M99 P15`. In the Haas control, a `GOTO` command can be used on the same line as other G-codes. The `GOTO` is executed after any other commands like M codes.

Computed Branch (GOTO#n and GOTO [expression])

Computed branching allows the program to transfer control to another line of code in the same subprogram. The control can compute the block while the program runs, using the `GOTO [expression]` form, or it can pass the block in through a local variable, as in the `GOTO#n` form.

The `GOTO` rounds the variable or expression result that is associated with the Computed branch. For instance, if variable #1 contains 4.49 and the program contains a `GOTO#1` command, the control attempts to transfer to a block that contains `N4`. If #1 contains 4.5, then the control transfers to a block that contains `N5`.

Example: You could develop this code skeleton into a program that adds serial numbers to parts:

```
%
O50002 (COMPUTED BRANCHING) ;
(D=Decimal digit to engrave) ;
;
IF [[#7 NE #0] AND [#7 GE 0] AND [#7 LE 9]] GOTO99 ;
#3000=1 (Invalid digit) ;
;
N99;
#7=FIX[#7] (Truncate any fractional part) ;
;
GOTO#7 (Now engrave the digit) ;
;
N0 (Do digit zero) ;
M99 ;
;
N1 (Do digit one) ;
;
M99 ;
%
```

With the above subprogram, you would use this call to engrave the fifth digit:

G65 P9200 D5 ;

Computed GOTOS using expression could be used to branch processing based on the results of reading hardware inputs. For example:

```
%  
GOTO [[#1030*2]+#1031] ;  
N0(1030=0, 1031=0) ;  
...M99 ;  
N1(1030=0, 1031=1) ;  
...M99 ;  
N2(1030=1, 1031=0) ;  
...M99 ;  
N3(1030=1, 1031=1) ;  
...M99 ;  
%
```

#1030 and #1031.

Conditional Branch (IF and M99 Pnnnn)

Conditional branching allows the program to transfer control to another section of code within the same subprogram. Conditional branching can only be used when macros are enabled. The Haas control allows two similar methods for accomplishing conditional branching:

```
IF [<conditional expression>] GOTO n
```

As discussed, <conditional expression> is any expression that uses any of the six Boolean operators EQ, NE, GT, LT, GE, or LE. The brackets surrounding the expression are mandatory. In the Haas control, it is not necessary to include these operators. For example:

```
IF [#1 NE 0.0] GOTO5 ;
```

could also be:

```
IF [#1] GOTO5 ;
```

In this statement, if the variable #1 contains anything but 0.0, or the undefined value #0, then branching to block 5 occurs; otherwise, the next block is executed.

In the Haas control, a <conditional expression> is also used with the M99 Pnnnn format. For example:

```
G00 X0 Y0 [#1EQ#2] M99 P5;
```

Here, the conditional is for the M99 portion of the statement only. The machine tool is instructed to go to X0, Y0 whether or not the expression evaluates to True or False. Only the branch, M99, is executed based on the value of the expression. It is recommended that the IF GOTO version be used if portability is desired.

Conditional Execution (IF THEN)

Execution of control statements can also be achieved by using the IF THEN construct. The format is:

```
IF [<conditional expression>] THEN <statement> ;
```



NOTE:

To preserve compatibility with FANUC syntax THEN may not be used with GOTO.

This format is traditionally used for conditional assignment statements such as:

```
IF [#590 GT 100] THEN #590=0.0 ;
```

Variable #590 is set to zero when the value of #590 exceeds 100.0. In the Haas control, if a conditional evaluates to FALSE (0.0), then the remainder of the IF block is ignored. This means that control statements can also be conditioned so that we could write something like:

```
IF [#1 NE #0] THEN G01 X#24 Y#26 F#9 ;
```

This executes a linear motion only if variable #1 has been assigned a value. Another example is:

```
IF [#1 GE 180] THEN #101=0.0 M99 ;
```

This says that if variable #1 (address A) is greater than or equal to 180, then set variable #101 to zero and return from the subprogram.

Here is an example of an IF statement that branches if a variable has been initialized to contain any value. Otherwise, processing continues and an alarm is generated. Remember, when an alarm is generated, program execution is halted.

```
%  
N1 IF [#9NE#0] GOTO3 (TEST FOR VALUE IN F) ;  
N2 #3000=11 (NO FEED RATE) ;  
N3 (CONTINUE) ;  
%
```

Iteration/Looping (WHILE DO END)

Essential to all programming languages is the ability to execute a sequence of statements a given number of times or to loop through a sequence of statements until a condition is met. Traditional G coding allows this with the use of the L address. A subprogram can be executed any number of times by using the L address.

```
M98 P2000 L5 ;
```

This is limited since you cannot terminate execution of the subprogram on condition. Macros allow flexibility with the WHILE-DO-END construct. For example:

```
%  
WHILE [<conditional expression>] DOn ;  
<statements> ;  
ENDn ;  
%
```

This executes the statements between DOn and ENDn as long as the conditional expression evaluates to True. The brackets in the expression are necessary. If the expression evaluates to False, then the block after ENDn is executed next. WHILE can be abbreviated to WH. The DOn-ENDn portion of the statement is a matched pair. The value of n is 1-3. This means that there can be no more than three nested loops per subprogram. A nest is a loop within a loop.

Although nesting of WHILE statements can only be up to three levels, there really is no limit since each subprogram can have up to three levels of nesting. If there is a need to nest to a level greater than 3, then the segment containing the three lowest levels of nesting can be made into a subprogram thus overcoming the limitation.

If two separate WHILE loops are in a subprogram, they can use the same nesting index. For example:

```

%
#3001=0 (WAIT 500 MILLISECONDS) ;
WH [#3001 LT 500] DO1 ;
END1 ;
<Other statements>
#3001=0 (WAIT 300 MILLISECONDS) ;
WH [#3001 LT 300] DO1 ;
END1 ;
%
```

You can use `GOTO` to jump out of a region encompassed by a `DO-END`, but you cannot use a `GOTO` to jump into it. Jumping around inside a `DO-END` region using a `GOTO` is allowed.

An infinite loop can be executed by eliminating the `WHILE` and expression. Thus,

```

%
DO1 ;
<statements>
END1 ;
%
```

executes until the `RESET` key is pressed.



CAUTION:

The following code can be confusing:

```

%
WH [#1] DO1 ;
END1 ;
%
```

In this example, an alarm results indicating no `Then` was found; `Then` refers to the `D01`. Change `D01` (zero) to `DO1` (letter O).

6.7.6 Communication With External Devices - DPRNT[]

Macros allow additional capabilities to communicate with peripheral devices. With user provided devices you can digitize parts, provide runtime inspection reports, or synchronize controls.

Formatted Output

The `DPRNT` statement lets programs send formatted text to the serial port. `DPRNT` can print any text and any variable to the serial port. The form of the `DPRNT` statement is as follows:

```
DPRNT [<text> <#nnnn[wf]>... ] ;
```

`DPRNT` must be the only command in the block. In the previous example, `<text>` is any character from A to Z or the letters (+,-,/,*, and the space). When an asterisk is output, it is converted to a space. The `<#nnnn[wf]>` is a variable followed by a format. The variable number can be any macro variable. The format `[wf]` is required and consists of two digits within square brackets. Remember that macro variables are real numbers with a whole part and a fractional part. The first digit in the format designates the total places reserved in the output for the whole part. The second digit designates the total places reserved for the fractional part. The control can use any number from 0-9 for both whole and fractional parts.

A decimal point is printed out between the whole part and the fractional part. The fractional part is rounded to the least significant place. When zero places are reserved for the fractional part, then no decimal point is printed out. Trailing zeros are printed if there is a fractional part. At least one place is reserved for the whole part, even when a zero is used. If the value of the whole part has fewer digits than have been reserved, then leading spaces are output. If the value of the whole part has more digits than has been reserved, then the field is expanded so that these numbers are printed.

The control sends a carriage return after every `DPRNT` block.

`DPRNT[]` Example:

Code	Output
#1= 1.5436 ;	
DPRNT[X#1[44]*Z#1[03]*T#1[40]] ;	X1.5436 Z 1.544 T 1
DPRNT[***MEASURED*INSIDE*DIAMETER** *] ;	MEASURED INSIDE DIAMETER
DPRNT[] ;	(no text, only a carriage return)

Code	Output
#1=123.456789 ;	
DPRNT[X-#1[35]] ;	X-123.45679 ;

DPRNT[] Settings

Setting 261 determines the destination for DPRNT statements. You can choose to output them to a file, or to a TCP port. Settings 262 and 263 specify the destination for DPRNT output. Refer to the Settings section of this manual for more information.

Execution

DPRNT statements are executed at look-ahead time. This means that you must be careful about where the DPRNT statements appear in the program, particularly if the intent is to print out.

G103 is useful for limiting lookahead. If you wanted to limit look-ahead interpretation to one block, you would include this command at the start of your program: This causes the control to look ahead (2) blocks.

```
G103 P1 ;
```

To cancel the lookahead limit, change the command to G103 P0. G103 cannot be used when cutter compensation is active.

Editing

Improperly structured or improperly placed macro statements will generate an alarm. Be careful when editing expressions; brackets must be balanced.

The DPRNT[] function can be edited much like a comment. It can be deleted, moved as a whole item, or individual items within the bracket can be edited. Variable references and format expressions must be altered as a whole entity. If you wanted to change [24] to [44], place the cursor so that [24] is highlighted, enter [44] and press **[ENTER]**. Remember, you can use the jog handle to maneuver through long DPRNT[] expressions.

Addresses with expressions can be somewhat confusing. In this case, the alphabetic address stands alone. For instance, this block contains an address expression in X:

```
G01 X [COS [90]] Z3.0 (CORRECT) ;
```

Here, the X and brackets stand-alone and are individually editable items. It is possible, through editing, to delete the entire expression and replace it with a floating-point constant.

```
G01 X 0 Z3.0 (WRONG) ;
```

The above block will result in an alarm at runtime. The correct form looks as follows:

```
G01 X0 Z3.0 (CORRECT) ;
```



NOTE:

There is no space between the X and the Zero (0). REMEMBER when you see an alpha character standing alone it is an address expression.

6.7.7 G65 Macro Subprogram Call Option (Group 00)

G65 is the command that calls a subprogram with the ability to pass arguments to it. The format follows:

```
G65 Pnnnnn [Lnnnn] [arguments] ;
```

Arguments italicized in square brackets are optional. See the Programming section for more details on macro arguments.

The G65 command requires a P address corresponding to a program number currently located in the control's drive or path to a program. When the L address is used the macro call is repeated the specified number of times.

When a subprogram is called, the control looks for the subprogram on the active drive or the path to the program. If the subprogram cannot be located on the active drive, the control looks in the drive designated by Setting 251. Refer to the Setting Up Search Locations section for more information on subprogram searching. An alarm occurs if the control does not find the subprogram.

In Example 1, subprogram 1000 is called once without conditions passed to the subprogram. G65 calls are similar to, but not the same as, M98 calls. G65 calls can be nested up to 9 times, which means, program 1 can call program 2, program 2 can call program 3 and program 3 can call program 4.

Example 1:

```
%
G65 P1000 (Call subprogram 001000 as a macro) ;
M30 (Program stop) ;
001000 (Macro Subprogram) ;
...
M99 (Return from Macro Subprogram) ;
%
```

In Example 2, the program LightHousing.nc is called using the path that it is in.

Example 2:

```
%
G65 P15 A1. B1.;
G65 (/Memory/LightHousing.nc) A1. B1.;
```



NOTE:

Paths are case sensitive.

6.7.8 Aliasing

Aliased codes are user defined G and M-codes that reference a macro program. There are 10 G alias codes and 10 M alias codes available to users. Program numbers 9010 through 9019 are reserved for G-code aliasing and 9000 through 9009 are reserved for M-code aliasing.

Aliasing is a means of assigning a G-code or M-code to a G65 P##### sequence. For instance, in the previous Example 2, it would be easier to write:

```
G06 X.5 Y.25 Z.05 F10. T10 ;
```

When aliasing, variables can be passed with a G-code; variables cannot be passed with an M-code.

Here, an unused G-code has been substituted, G06 for G65 P9010. In order for the previous block to work, the value associated with subprogram 9010 must be set to 06. Refer to the Setting Aliases section for how to setup aliases.



NOTE:

G00, G65, G66, and G67 cannot be aliased. All other codes between 1 and 255 can be used for aliasing.

If a macro call subprogram is set to a G-code and the subprogram is not in memory, then an alarm is given. Refer to the **G65 Macro Subprogram Call** section on page **240** on how to locate the subprogram. An alarm occurs if the sub-program is not found.

Setting Aliases

The G-code or M-code alias setup is done on the Alias Codes window. To setup an alias:

1. Press **[SETTING]** and navigate to the **Alias Codes** tab.
2. Press **[EMERGENCY STOP]** on the control.
3. Using the cursor keys select the M or G Macro Call to be used.
4. Enter the number of the G-code or M-code you want to alias. For example, if you want to alias G06 type 06.
5. Press **[ENTER]**.
6. Repeat steps 3 - 5 for other aliased G or M-codes.
7. Release the **[EMERGENCY STOP]** on the control.

Setting an alias value to 0 disables aliasing for the associated subprogram.

F6.10: Alias Codes Window

Settings And Graphics	
Graphics	Settings
Network	Notifications
Rotary	Alias Codes
M-Codes & G-Codes Program Aliases	
	Value
M MACRO CALL 09000	0
M MACRO CALL 09001	0
M MACRO CALL 09002	0
M MACRO CALL 09003	0
M MACRO CALL 09004	0
M MACRO CALL 09005	0
M MACRO CALL 09006	0
M MACRO CALL 09007	0
M MACRO CALL 09008	0
M MACRO CALL 09009	0
G MACRO CALL 09010	0
G MACRO CALL 09011	0
G MACRO CALL 09012	0
G MACRO CALL 09013	0
G MACRO CALL 09014	0
G MACRO CALL 09015	0
G MACRO CALL 09016	0
G MACRO CALL 09017	0
G MACRO CALL 09018	0
G MACRO CALL 09019	0